

Lecture 12

Physics-Informed Neural Networks for Geometry I

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Setembro 2026

Overview

1. Why Neural PDE Solvers?
2. The PINN Framework
3. Worked Example & Geometric PDEs
4. Reading the Failure Modes
5. Summary

Recap (Lecture 11): Manifold architectures, diffusion models for generating geometric examples.
Guiding question: *How can a neural architecture serve as the trial function of a PDE solver?*

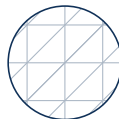
Classical Numerics and Its Pressure Points

The classical toolbox

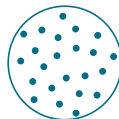
Finite differences (grids), finite elements (meshes), spectral methods (global bases):

- Mature methods with convergence theory.
- Strong baselines for standard forward PDEs in two or three dimensions.

- **Geometry:** moving domains are hard to mesh, so preferable to train on sampled points.
- **High dimension:** grids grow rapidly with dimension; neural ansätze can exploit structure in the solution, when it is present.
- **Inverse problems:** unknown coefficients or geometry can be learned jointly with a solution from partial observations.
- **Parametric families:** include parameters as inputs, or learn a solution operator (L13), to share work across related problems.



mesh: connected elements



collocation: sampled points

AI approach benefits

- Mesh-free.
- Training cost scales *much* better with dimension.
- Flexible differentiable trial functions.
- Error control often more stable.

PINNs: Networks as Trial Functions

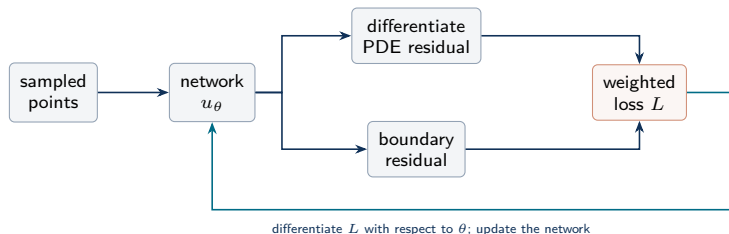
Fit the equation and the boundary conditions

For $\mathcal{N}[u] = f$ in Ω and $\mathcal{B}[u] = g$ on $\partial\Omega$, choose a neural trial function u_θ and minimise

$$L(\theta) = \lambda_r L_{\text{res}} + \lambda_b L_{\text{bc}} \quad (+ \lambda_d L_{\text{data}}),$$

$$L_{\text{res}} = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{N}[u_\theta](x_i) - f(x_i)|^2, \quad L_{\text{bc}} = \frac{1}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}[u_\theta](y_j) - g(y_j)|^2.$$

The **collocation points** $x_i \in \Omega$ and $y_j \in \partial\Omega$ are where we test the equations.



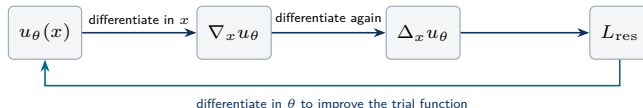
- The λ weightings balance the objectives, something to tune / schedule.
- Optional L_{data} fits any observed solution values with a traditional supervised loss.

Autodiff Builds the Residual

Two uses of the chain rule

Automatic differentiation computes derivatives of:

- 1 Outputs (u_θ) with respect to inputs (x), to build the PDE residual.
- 2 The loss (L) with respect to parameters (θ), to train.



- **Derivative accuracy:** No finite-difference stencil error; such that derivatives are evaluated up to floating-point error. Approximation and sampling errors remain.
- **Smoothness:** Use smooth activations, such as $\tanh/\text{GELU}/\text{SiLU} \Rightarrow u_\theta$ smooth! ReLU networks have zero classical second derivatives almost everywhere, making them unsuitable for $\geq 2\text{nd}$ order residuals (still poor for 1st order).
- **Cost:** Second derivatives require additional differentiation. In high dimensions, a stochastic trace estimator can approximate $\Delta u = \text{tr} \nabla^2 u$.
- **Sampling:** The empirical loss estimates an integrated squared residual. Add collocation points where the residual is large to resolve difficult regions.

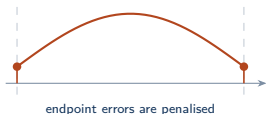
Boundary Conditions: Soft vs Hard

Soft: add a penalty

Extra loss term:

$$L = \lambda_{\text{res}} L_{\text{res}} + \lambda_b L_{\text{bc}}.$$

- Flexible across domains and boundary-condition types.
- Non-zero error introduced.
- Boundary error competes with residual error (λ_b needs training resources).



- On the unit disk with zero boundary data, take $\tilde{g} = 0$, $\varphi(x) = 1 - \|x\|^2$.
- Spectral features enforce periodicities; e.g. $(\cos x, \sin x)$ enforces 2π -periodicity.
- Use hard constraints when feasible, and report boundary error separately.

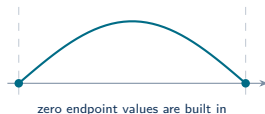
Hard: build into ansatz

E.g. for Dirichlet:

$$u_\theta = \tilde{g} + \varphi N_\theta.$$

...for $\tilde{g}|_{\partial\Omega} = g$ and $\varphi|_{\partial\Omega} = 0$.

- Boundary conditions hold exactly, by construction.
- More technical implementation: requires a usable extension $\tilde{g}$ and smooth vanishing factor φ .



Worked Example: Poisson on the Unit Disk

A manufactured solution gives a known answer

On $\mathbb{D}$, solve $-\Delta u = 4 + 8x_1$, with $u|_{\partial\mathbb{D}} = 0$. The exact solution is $u^*(x) = (1 - \|x\|^2)(1 + x_1)$.

net: smooth MLP; x: batch of points in the disk

```
def u(x):
```

```
    phi = 1 - x.square().sum(1, keepdim=True)
```

```
    return phi * net(x)
```

```
def derivative(y, x):
```

```
    return torch.autograd.grad(
```

```
        y.sum(), x, create_graph=True)[0]
```

```
x.requires_grad_(True)
```

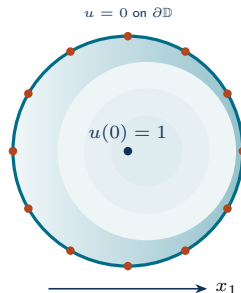
```
du = derivative(u(x), x)
```

```
lap = sum(derivative(du[:, k], x)[:, k]
```

```
    for k in range(2))
```

```
forcing = 4 + 8*x[:, 0:1]
```

```
loss = ((-lap - forcing)**2).mean()
```



- Train with Adam; optionally finish with L-BFGS, a quasi-Newton optimiser.
- Resample interior points with each batch during training.
- Measure relative L^2 error on independent test points.
- Compare accuracy and runtime against a classical numerical solve.

Geometric PDEs: Laplace–Beltrami and Friends

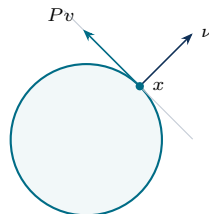
Riemannian differential operators

- For PDEs on manifolds on $(\mathcal{M}, g)$, the differential operator $\mathcal{N}$ of the residual-loss L_{res} uses updated operators $\{\Delta \mapsto \Delta_g\}$. Use the sign convention $\Delta_g = \operatorname{div}_g \operatorname{grad}_g$.
- Sample with respect to $\mathcal{M}$, via volume element $\sqrt{\det(g)}$.

- **Discrete geometry:** mesh or point-cloud Laplacians are another approximation; check discretisation error.
- **Embedded architectures:** restrict an ambient network to $\mathcal{M}$ and use tangential derivatives $P\nabla$. For unit normal ν , the projector is $P = I - \nu\nu^\top$.
- **Atlas architectures:** use the coordinate formula

$$\Delta_g u = \frac{1}{\sqrt{|g|}} \partial_i \left(\sqrt{|g|} g^{ij} \partial_j u \right),$$

and match chart predictions on overlaps.



differentiate along $\mathcal{M}$

Geometric targets

- Surface eigenfunctions $-\Delta_g u = \lambda u$.
- Harmonic maps $\Phi : (\mathcal{M}, g) \rightarrow (\mathcal{N}, h)$.
- Conformal factors $\tilde{g} = e^{2\phi} g$.

Autodiff can differentiate through the metric, tangential projectors, etc.

Variational Cousins and Flow Previews

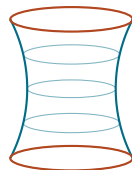
Deep Ritz: Minimising an Energy

For solving the PDE: $-\Delta u = f$ with suitable boundary constraints, instead minimise the respective energy:

$$E[u_\theta] = \int_{\Omega} \left(\frac{1}{2} \|\nabla u_\theta\|^2 - f u_\theta \right) dx.$$

- Its Euler–Lagrange equation is the PDE.
- Monte Carlo integration needs only first derivatives here, instead of a Laplacian residual.

- **Minimal surfaces:** parameterise an immersion X_θ and minimise its area, with fixed boundary curves.
- **Prescribed curvature:** fit a residual such as $H[X_\theta] - h$ for an immersion X_θ .
- **Flows:** adding time, $X_\theta(x, t)$ leads to mean-curvature and other geometric evolutions (L13), with new questions of causality and singularities.



catenoid between fixed rings
(schematic)

Formulations for training objectives

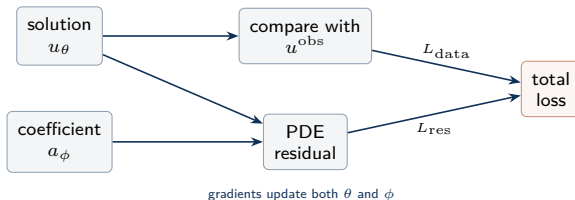
- Energy: reduces derivative order (cheap & stable), when a variational principle is available.
- Residuals: covers a wider range of equations, compare conditioning and actual solution error.

Inverse Problems: Learning What Is Unknown

Learning coefficients with the solution

Example: For the PDE $-\nabla \cdot (a \nabla u) = f$
...simultaneously learn the solution u and the conductivity $a(x) > 0$,
...assisted by observations of u (L_{data} supervised loss).
Parameterise u_θ and a_ϕ and minimise:

$$L(\theta, \phi) = \| -\nabla \cdot (a_\phi \nabla u_\theta) - f \|_\Omega^2 + \lambda_d \| u_\theta - u^{\text{obs}} \|_{\text{data}}^2 + \text{regularisers}.$$



- Generalisation: sources $f(x)$, potentials $V(x)$, metric g , general $\mathcal{N}$ coefficients, etc.
- Ambiguity: several coefficients may fit the same measurements. Extra experiments, boundary information, or regularisation may be essential.
- Validation: check on observations excluded from training; low training loss insufficient.

Training Pathologies: an Honest Preview

- **Loss imbalance:** The PDE fits while boundary values fail, or vice versa.
→ Inspect each loss term and its gradient separately; tune weightings.
- **Spectral bias:** Broad structure fits before fine oscillations or boundary layers.
→ Compare errors across spatial scales.
- **Stiffness:** Strongly contrasting scales make optimisation difficult.
→ Check nondimensionalisation and operator conditioning.
- **Propagation failure:** A time-dependent solution fits some times but misses full evolution
→ Inspect error as a function of time (L13) over full range.

Low residual does not certify a correct solution

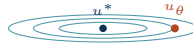
- Sparse samples, poor boundary fits, ill-conditioning can hide substantial error.
- Validate beyond the points, quantities, measures used in training.



sparse samples
matches the points,
not between them



poor boundary fit
residual can be small
while $u \neq 0$ at $\partial\Omega$



ill-conditioning
low loss can extend far
along a flat direction

Additional verification

- Check conservation laws, symmetries, boundary error, and compare with known solutions.
- Lecture 13 develops strategies for the failure modes above.

Summary

Key points

- PINNs combine neural trial functions with PDE and boundary losses at sampled points; autodiff supplies the required derivatives for operators and training.
- PINN functional flexibility is useful for complex geometry and inverse problems; ...benchmark speed and accuracy against classical numerical methods.
- Build boundary conditions into the ansatz when practical; ...use suitable smoothness for the differential operator.
- Consider embedding or atlas architectures for geometric PDEs; ...Deep Ritz learns through minimising an energy when one is available.
- Diagnose loss imbalance, spectral bias, stiffness, and propagation failure; ...validate the solution independently of its training loss.

Next

Tutorial 12: Poisson on a disk and a surface, compared with classical baselines.

Lecture 13: Time-dependent and nonlinear flows, training remedies, neural operators.